

Futures as prelude: Bitcoin price forecasting from perpetual futures data

B. Kachnowski

April 22, 2022

Working paper

Abstract

Crypto currency perpetual futures asset price changes seem to have some predictive power for short term (tens of minutes) forecasts of underlying crypto currency price changes. In this study, we apply simple vector auto-regressive models to Bitcoin spot and futures price changes and allow for a forecasting dead zone to discard forecasts near zero price change, with "near" being a tunable parameter. We have noted in-sample results that approach 70% directionally correct without extensive model refinement, with determinate forecasts occurring on a mean interval of 40 minutes or so. Such levels of accuracy and forecast intervals might be useful for guiding BTC trading. A simple out-of-sample check by a 90/10 time-ordered train/test split yielded similar accuracy as in-sample fit when the dead zone method was applied. Additional out-of-sample testing and verification of this method would probably be required before it could be used reliably in a trade recommendation system.

Introduction

The financial instrument of the perpetual futures contract [Ref 1], (apparently?) originally proposed by Robert Shiller in 1992, has achieved actively traded implementation in the cryptocurrency market of 2022 [Ref 2]. While as of this writing there are various constraints in the laws of the United States at the federal and state levels limiting who can trade such contracts-- and we speculate here, but such limits might be for investor protection from fraud and misunderstandings, coupled with possibly ill-understood risk profiles of these new financial instruments-- [Note 2] (and such legal limits may exist in other countries as well), it is useful to study the behavior of the trading prices of these contracts to gain a better understanding of how these futures assets might fit into an investment portfolio or automated trading system when such trades are allowed by law. As we will show, it is not necessary to actually trade the futures assets in order to take advantage of their data. Sub-markets that are not yet efficient (in an Efficient Market Hypothesis sense) and are not yet widely traded are *possibly* where profit can be had by specially-informed investors. As such, we undertook a small study of cryptocurrency futures prices as to how they might relate to or be predictive of the underlying asset prices. For example, if the futures price of an asset just went up, what do we expect the related spot price to do in the next time period?

Data pre-processing

For data sources, we use the tremendous volume of archived historical data available from the Binance cryptocurrency exchange, which provides futures prices by the millisecond time step, along with related underlying asset prices by the same time step [Ref 3]. Crypto prices may not be the same across exchanges, especially at the millisecond intervals we are looking at here, and so the study in this paper may not be relevant for other exchanges until verified otherwise. For this study, we focus on the *perpetual futures* assets provided for trading by Binance: futures contracts that never expire. In particular, we use the "aggTrades" (aggregate trades) data sets for this study [Note 3]. Since the data is at millisecond time steps and we are not interested in

that level of granularity at the moment, we aggregate the data using 10 minute non-overlapping intervals and take the simple average of all data points presented by Binance within these 10 minute intervals, not yet accounting for volume traded at each of the raw data points. Hence, we compress megabytes of millisecond data into simple time series data at 10 minute intervals. We do this for both the spot price of a given asset and the related USD-M (Perpetual) Futures data for the same asset. Readers can consult the Binance web site for the particular details of the USD-M futures product. Here we are not interested in the intricacies of the futures product itself, but how its price behaves over time.



Figure 0: Raw price data spot versus perpetual futures (USD-M) for BTC, March 1 to December 31 2021, 10 minute intervals. Two time series are drawn, but they seem as one.

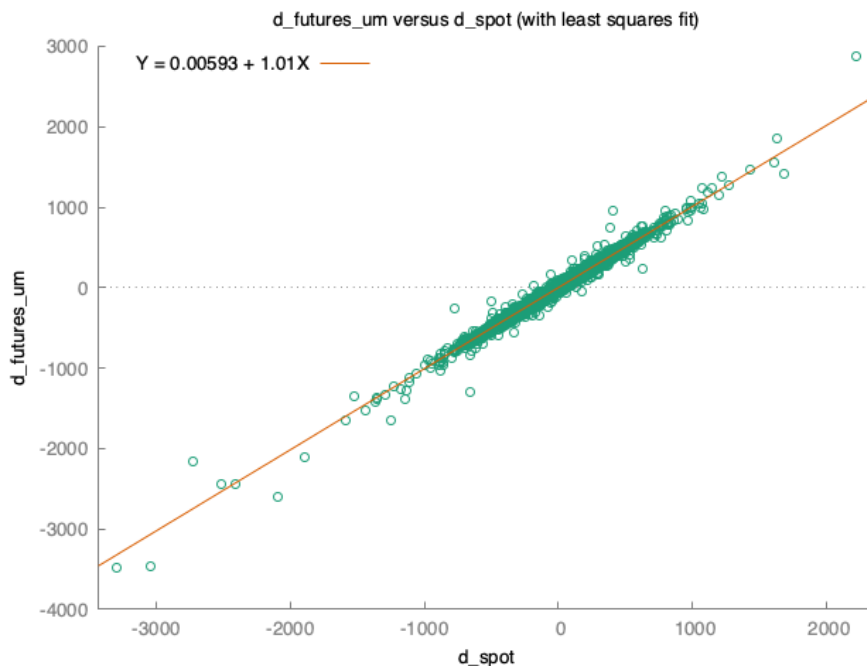


Figure 1: 10 minute change in spot price versus 10 minute change in futures price, same data as Figure 0.

The first observation of note is that perpetual futures prices in this interval for BTC seem to track the underlying spot prices very closely in the recent time periods we studied [Note 5]. Note that in Figure 0, these two price series are just about indistinguishable from each other by eye. While this may be obvious to

futures traders ("of course it does this! we knew it would do this twenty years ago!"), it was not obvious to the present author. Indeed, if a derivative asset so closely tracks the underlying, what is the point of trading the derivative? "Just trade the underlying and save yourself some confusion; the difference between spot and futures prices is so small, it is just noise in your portfolio." Such might be a counter-argument to the ideas presented in this paper. But the story is not complete of course (as it rarely is in these types of studies). Might the small differences in spot versus futures price have some predictive power? This is one of the questions we consider here.

When we go to first differences of prices (change in price over 10 minute interval for each asset as shown in Figure 1), the small deltas between the spot price change and the futures price change are a little more visible by inspection. We see slight scattering about the visible regression line, along with distinct points. Note that this is contemporaneous (at the same time step) correlation which we show mainly to indicate that, in Figure 0, while the spot and futures price is "the same" visually, there are small but non trivial differences which may be of use.

Contemporaneous correlation is not directly useful for the type of predictions we are trying to make here, so we start with the tried and true (linear, at least at first) vector autoregression method using the Gretl econometrics software [Ref 4] to see if the data might tell us something non-obvious, wherein prior data is used to forecast future price changes. Due to some of the issues with missing data which we discuss below, we found that the Ordinary Least Squares (OLS) regression method of Gretl was more effective at building models (with lag variables being constructed as a pre-processing step) from our data rather than the formal VAR (Vector Auto Regression) method in Gretl, with data points having missing data being automatically

left out of the OLS model by the software. These issues with using the Gretl VAR package directly might be resolved when we address the missing data in a less casual manner.

For initial proof of concept (not shown here), we prepared some time series data from the February/March 2022 period and constructed some models for BTC and ETH, throwing in the "meme" coin DOGE for good measure in some models. One and two month test models seemed to show some promise in terms of Adjusted R-square, with statistically significant coefficients and models typically involving 1 or 2 interval lags showing as relevant forecasters, with intervals being 10 minutes as noted above. Due to the positive results from these casual studies, we extended the study to a longer time period as described here.

For the first model we present here, we limit ourselves to BTC only data (spot and future, two time series) and use the time period March to December 2021. Our Binance data extract scripts failed for January and February 2021 for reasons that we have not investigated yet (was the data not available? was there a change in data format? etc?) so we left out those two months from this 2021 model for now.

Using the time delta in our averaged data of 10 minutes, we formed autoregressive models of BTC and its related USD(S)-M futures price 10 minute difference variables [Note 4] using some part-manual model refinement available in the Gretl software, and arrived at this model:

Model 1: OLS, using observations 1-43581 (T = 42188)

Missing or incomplete observations dropped: 1393

Dependent variable: d_spot

	coefficient	std. error	t-ratio	p-value	

const	0.00487120	0.670667	0.007263	0.9942	
d_spot_1lag	-1.29836	0.0493545	-26.31	2.68e-151	***
d_spot_2lag	-0.700886	0.0565410	-12.40	3.16e-35	***
d_spot_3lag	-0.260626	0.0549856	-4.740	2.15e-06	***
d_spot_4lag	-0.245064	0.0453929	-5.399	6.75e-08	***
d_futures_um_1lag	1.48815	0.0479414	31.04	3.44e-209	***
d_futures_um_2lag	0.635689	0.0559679	11.36	7.47e-30	***
d_futures_um_3lag	0.286308	0.0546179	5.242	1.60e-07	***
d_futures_um_4lag	0.253209	0.0456740	5.544	2.98e-08	***
Mean dependent var	-0.028559	S.D. dependent var	142.9849		
Sum squared resid	8.00e+08	S.E. of regression	137.7527		
R-squared	0.072023	Adjusted R-squared	0.071847		
F(8, 42179)	409.2031	P-value(F)	0.000000		
Log-likelihood	-267653.0	Akaike criterion	535324.0		
Schwarz criterion	535401.8	Hannan-Quinn	535348.5		

Listing 1: Baseline model BTC change in average spot price in 10 minutes as a function of prior changes in (spot prices and futures prices).

This is a first pass model and so we do not go to great extents yet to see if additional lags are significant, because: there are other variables we will want to add such as similar data from other crypto currencies; we may want to tune the time delta of 10 minutes; etc. Here we see that all four 10 minute lags from both spot and futures price are reporting as statistically significant predictors, e.g. low standard error versus coefficient, also noted by the very low p-values of each of these variables. Contrarily, the constant of the model is reporting with a very high p-value, indicating that the error of this constant is such that the constant should probably be considered to be zero (e.g., no bullish or bearish bias); the standard error of 0.67 overwhelms the constant itself of 0.0048, so it may be best to leave this out of the model. This we do and show in Listing 2:

Model 2: OLS, using observations 1-43581 (T = 42188)
Missing or incomplete observations dropped: 1393
Dependent variable: d_spot

	coefficient	std. error	t-ratio	p-value	
d_spot_1lag	-1.29836	0.0493538	-26.31	2.66e-151	***
d_spot_2lag	-0.700886	0.0565403	-12.40	3.16e-35	***
d_spot_3lag	-0.260626	0.0549849	-4.740	2.14e-06	***
d_spot_4lag	-0.245064	0.0453923	-5.399	6.75e-08	***
d_futures_um_1lag	1.48815	0.0479407	31.04	3.40e-209	***
d_futures_um_2lag	0.635688	0.0559672	11.36	7.46e-30	***
d_futures_um_3lag	0.286308	0.0546172	5.242	1.60e-07	***
d_futures_um_4lag	0.253209	0.0456735	5.544	2.98e-08	***
Mean dependent var	-0.028559	S.D. dependent var	142.9849		
Sum squared resid	8.00e+08	S.E. of regression	137.7510		
Uncentered R-squared	0.072023	Centered R-squared	0.072023		
F(8, 42180)	409.2130	P-value(F)	0.000000		
Log-likelihood	-267653.0	Akaike criterion	535322.0		
Schwarz criterion	535391.2	Hannan-Quinn	535343.8		

Listing 2: Remove insignificant constant term from the model of Listing 1 and re-fit remaining coefficients

Note that the R-square value stays the same when we eliminate this insignificant constant, indicating that it was a good idea to eliminate it. Actual versus predicted of this model is shown in Figure 3. While neither the R-square nor the associated Figure 3 seems very impressive at first, we will demonstrate that this model may have some value nonetheless.

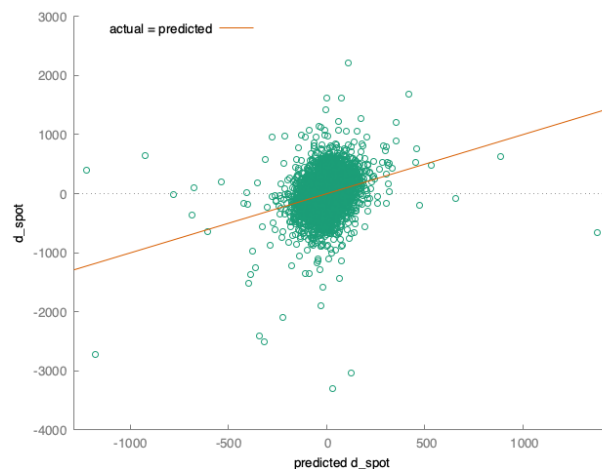


Figure 3: In-sample actual versus predicted for the model in Listing 2 (no coefficient, 4 lags each of `d_spot` and `d_futures_um`). Model forecast is on horizontal axis.

Missing data, initial concerns

This 10 month model is of course a first pass model and has the issue of missing data. E.g. during some of the 10 minute intervals, there was no price data available for the futures asset, probably due to no trades occurring for that asset during the 10 minute averaging window, but possibly (unlikely?) due to some data gathering glitch in the Binance systems.

This is handled at first automatically by the "join" operation of Gretl which matches data via time step, and we achieve timestamp-reconciled data sets such as this snippet in the following Table 1. The NA symbol indicates that there is no data at the particular time step for the `futures_um` time series.

i	timestamp	spot	futures_um
148	1614646200	49577.799798942	49588.5217210868
149	1614646800	49423.5910351249	49450.1924163021
150	1614647400	49176.0402949953	NA
151	1614648000	48947.8544074943	NA
152	1614648600	49093.8278165986	NA
153	1614649200	49027.6466175934	NA
154	1614649800	49193.0504432758	NA
155	1614650400	49325.598631579	49391.3566326531
156	1614651000	49279.7177786623	49303.3360501285

Table 1: Example of of raw price data with missing values

Our test time period of Mar 1 to Dec 31 2021 has 43581 10-minute time periods, and there were 289 of these 10 minute time periods where the data is marked as missing for the futures_um data-- again, possibly indicating that no trades occurred during those 289 intervals for futures_um.

There are various methods to handle missing data in modeling, and we start by using what is readily available in the software we are applying, with the caveat that missing data handling might be improved upon as a model refinement. A limitation of the Gretl code we are using to build this model is that if we delete data points with one (or more) missing values, the time series is considered "broken" or non-contiguous, and Gretl no longer allows lags to be computed. So we compute lags first before deleting points. Some of the lagged values then are also NA (missing), and these new NA terms "pollute" additional data rows with missing data. Fortunately, the base OLS model builder in Gretl handles these missing data values automatically, and the number of data points dropped from the OLS fit are show at the top of the model output, viz:

Missing or incomplete observations dropped: 1393

This is more than the raw number of missing values (289) due to the lag operation propagating the missing values as "pollution" to new data rows as we noted above. In actual use forward in time in a trading recommender, if one or more of the values we feed into the model is NA or missing, we just would not generate a forecast for that time step.

As a reminder, these first two models model attempt to predict the 10 minute **change** in the 10 minute average price of BTC spot based on prior values, not the absolute price itself [Note 8].

Coefficient interpretation

A benefit of traditional econometric models versus black box machine learning models is that we can look at the model internals and see if they make sense or match intuition. This is of course a mixed benefit, because intuition is sometimes wrong in finance, physics, and a variety of other fields, and a bad model might bias our already wrong notions further in an incorrect direction. Setting these concerns aside temporarily, and examining the coefficients of the lags, we see that lag 1 (10 minutes prior) has the highest coefficient (in absolute value) for both spot and futures, and these coefficients decay as we go back in lag steps. E.g. the spot lag coefficients are, roughly:

-1.29, -0.70, -0.26, -0.24

And the futures coefficients are:

1.49, 0.63, 0.29, 0.25

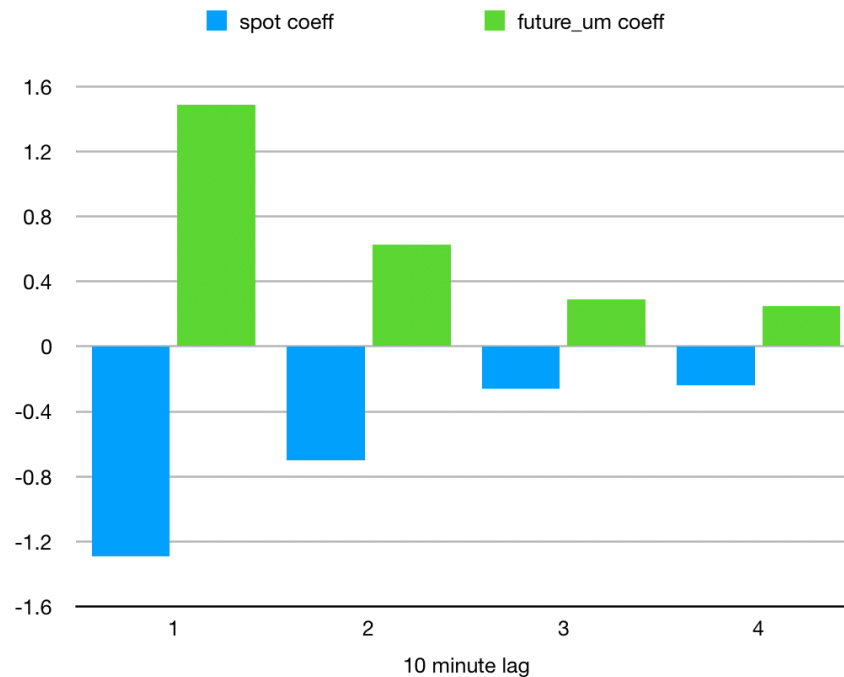


Figure 2: Model coefficients, showing decay as data gets older

Hence, the most recent price change has the most effect on the forecast, then the effect decays as we go back in time. Since spot and futures prices track very similarly in this time interval of study (March to December 2021), the absolute sizes of the coefficients are directly comparable without the need for data normalization.

If we admit that there may be intuition of the markets, this matches the intuition of "more recent events have more effect on upcoming prices than older events that are already 'priced in' to an asset."

For example, since the four lagged spot price coefficients have negative values, it indicates a kind of oscillatory or perhaps mean-reverting effect in the next 10 minute period: If BTC had been going down recently, it will have a tendency to go up in the next interval, and vice-versa. Similarly, the futures prices have a reinforcing or "momentum" [Note 7] effect upon the spot price of BTC: if recent futures price changes were up, it indicates that BTC spot price will go up. Now we must take care to re-examine the low-ish R-square value of 0.072, also looking at the mostly random scatter of the actual versus fit dot plot of Figure 3: these oscillation and reinforcing effects are not strong, they are "light and gentle." There are many other things going on in this data. Looking at the R-square, this is about a 7% effect (the remaining 93% of the spot price changes not being accounted for by this model). Nonetheless, as we will see, the 7% is only the raw model effect, and we should be able to improve upon real-world interpretation of this by considering what we call a dead zone tolerance. Furthermore, in an

efficient market, we would expect to see *zero* effect of prior changes on future changes, so a 7% effect with 8 strong predictor variables seems to be worth looking into.

While the Adjusted R-square of this model is low, it is not zero, and the low error versus coefficient (hence, exceedingly small p-values) indicate that the model may indeed have uncovered something of interest to an automated trading system.

Other possible models and potential use

Note that here we are only concerned with forecasting the spot price change of BTC. We could also try to forecast the 10 minute price change of the futures asset itself using the same lagged price change methods, and casual trials of futures price change forecasting showed promise. For this futures forecasting to be useful, we would have to be allowed by law to trade these futures products. This may not be available to all traders at this writing as we noted earlier. However, an interesting part of models like this is that if we do not want to invest in futures ourselves or cannot invest in these futures products (due to legal limits, lack of understanding, etc), we can still use futures *data* in a model like this to try to inform us how to trade the underlying asset itself. The fact that there is an active futures market out there (whether or not we care to or are allowed to trade it) can *inform* our trades in ordinary crypto assets themselves, as long as companies like Binance provide historical and timely market data upon which we can build these type of models.

Dead zone refinement

As described in our related white paper [Ref 5], applying a dead zone around a zero-centered forecast might be useful for improving the percent directionally correct forecasts of a regression model that tries to forecast a floating point value. In short, we discard forecasts that are "too close" to zero and only count forecasts outside the dead zone for accuracy checks, with "too close" being a tunable parameter. Here we provide only an example of this rather than doing extensive tuning of the dead zone, since model tuning should typically be done with a selection of parameters at once, rather than one parameter at a time. It is a safe assumption that model hyper-parameters such as this dead zone tolerance are non-linear with respect to trade/no-trade signals (even though the model itself in this case is linear), and the hyper-parameters may interact with one another in non-obvious manners.

dead zone (in dollars)	total points outside dead zone	directionally correct forecasts	% correct direction
0	42188	24939	59.1%
10	29019	17971	61.9%
20	18504	11959	64.6%
30	11379	7561	66.4%
40	7048	4835	68.6%
50	4463	3179	71.2%

Table 2: Example effect of dead zone on percent correct for this model

Applying a dead zone reduces the number of forecasts that could potentially be used for trade recommendations. E.g. instead of recommending a trade direction every 10 minutes, if we use a dead zone of, say, \$30, we only get $11379/42188 \approx 27\%$ of the total number of 10 minute time intervals with a forecasted direction. This is about $1/4$ of the total number of 10 minute intervals, so it suggests that we would get a trade direction recommendation from this model, on average, every 40 minutes or so. For the remaining time intervals, the forecast would be considered indeterminate, and we would not recommend a trade for those intervals (from this model). The important point to note here is that by taking into account this dead zone, we can start with a model with a seemingly weak R^2 of 7% and boost its directional correctness significantly (close to or above 70% accuracy in this example here) [Note 6].

Practical considerations

For this model, we are forecasting an *average* spot price in a 10 minute interval. Whether this can be effectively used to feed into trading system where, after all, you have to buy and sell at a particular price, not an average price, remains to be seen. In other words, if we use an average price forecast to inform a trading system, can that trading system be profitable? Additionally, this is an in-sample data analysis. An important next step will be to test how this model performs out-of-sample (the start of which is shown in Section 2 of this paper). Further, there are various model refinements that are possible: we could add additional predictors such as similar data from Ethereum (ETH) and other currencies into this model, we could refine how much data we put into the model (how many 10 minute intervals), we could tune the averaging interval for best results, we could use a trading volume weighting scheme to compute the average 10 minute prices, we could try to bring the VARIMA or equivalently VARMA [Ref 6] class of models into the picture (e.g. adding "MA = Moving Average" terms to the model; not the simple historical moving averages that are often plotted along with raw price data, but the more sophisticated implicit MA terms that add "stickiness" or "memory" to the model by feeding back forecasted results into the model for a self-correcting effect). We could also try to add fractionally differenced terms into this simple model, which allows more of the somewhat distant past (memory) of the time series to influence the forecast-- versus our use of simple differencing in this trial study which only takes into account the immediate past. All of this could be done before we apply advanced machine learning techniques to the problem, with the earlier noted benefit of visibility of the mechanisms of the model (e.g. simple algebraic terms that we can read and understand). But the important point of this test model is that even with two price data streams and a simple linear model, the model seems to be uncovering some intra-day predictability of the BTC spot price change. Additionally, practitioners will note that we have not checked the stationarity of the 10 minute price changes used in this introductory model study. However, we leave this to further studies of out-of-sample model performance: if the model performs well out-of-sample, it is less critical that the data is formally stationary. If the model does not perform well out-of-sample, we do not really care if the data is stationary or not, though making the data stationary might well assist in out-of-sample forecast improvement.

Another commonly applied step in these types of models is to analyze the model residuals (actual - model error) to see if we have "left any money on the table," vis-a-vis: Do the model residuals tell us that there may be more information left that we have not yet considered which we can use to improve accuracy? This is yet another model refinement that could be attempted.

Summary

Simple linear vector autoregression models of BTC price changes and the related perpetual futures price changes show promise for intra-day forecasting of these price changes, especially when we allow for a forecasting dead zone in which we discard forecasts that we classify as indeterminate. Models indicate that some prior data seem to have correlation to some future data, not necessarily in a pairwise manner, but through a multi variable regression. While additional out-of-sample testing and various types of model refinements / robustness enhancements are indicated before trying to build a trade recommendation system from a model like this, our results seem to indicate that such refinements might be valuable.

References and Notes

[Ref 1] Shiller, R., *Measuring Asset Values for Cash Settlement in Derivatives Markets: Hedonic Repeated Measures Indices and Perpetual Futures*, <https://cowles.yale.edu/sites/default/files/files/pub/d10/d1036.pdf>

[Ref 2] *Binance Frequently Asked Questions*, <https://www.binance.com/en/support/faq/d2a1afd5f829455c9ded23f0ca561a40>

[Ref 3] *Binance Historical Market Data*, <https://www.binance.com/en/landing/data>

[Ref 4] Cotrell, A., Luchetti, R., *Gnu Regression, Econometrics and Time-series Library*, <http://gretl.sourceforge.net/gretl-help/gretl-guide.pdf>, March 2022

[Ref 5] Kachnowski, B., *Returns Distribution Moments as One Day Ahead Forecasters of Crypto Currency Returns*, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3972810

[Ref 6] MATLAB documentation, <https://www.mathworks.com/help/econ/introduction-to-vector-autoregressive-var-models.html>

[Note 1] deleted

[Note 2] Indeed, the risk profiles of the underlying assets (the cryptocurrencies) themselves are often ill-understood, and the legal category of different types of cryptocurrencies (consider stable coins which try to track fiat currencies *versus* BTC which floats with respect to fiat, etc) is subject to ongoing scrutiny by legislative bodies the world over.

[Note 3] From [Ref 2] "[aggTrades] Combines the aggregate total volume of orders filled at a given price throughout daily or monthly intervals."

[Note 4] Per "best (initial) practice" of converting drifting non-stationary price series into more stationary difference series, and leaving the possibility open that fractional differencing may improve model quality.

[Note 5] We do not yet know if this behavior of perpetual futures prices tracking so closely to the underlying spot prices is universal for all crypto currencies or all time intervals.

[Note 6] Applying a dead zone to a regression model may not improve percent directionally correct results for all models.

[Note 7] The momentum effect here is not the momentum of the asset itself, but the tendency of the futures data to "push" the asset in the same direction that the futures data has been going recently (bullish or bearish). It is a cross-variable momentum effect.

[Note 8] A ten minute time interval may not be ideal for a trade recommender that is not connected to an automated trade system because it requires that a trade be executed very soon after the signal, and then the position closed out ten minutes later. Hence, we may want to try a similar study with longer time intervals on the forecast side (at least, if not on the predictor side).

Section 2: Out-of-sample first test

In this section, we perform a simple out-of-sample check of the model as follows:

- refit the model to the first 90% of our data set (the older data in our dual time series)
- use the model to forecast the final 10% of the data set (by time sequence, the most recent points)
- check the metrics of these 10% out-of-sample forecasts

This is typically referred to as a train/test split of the data in machine learning parlance, with the test data block following the training data block due to the time-ordered nature of this problem.

From the starting point count of 43581, we choose $43581 \times 0.9 = 39223$ points (some of which have missing data as noted previously) and re-fit the model using the same variables.

Model 7: OLS, using observations 1-39223 (T = 37870)

Missing or incomplete observations dropped: 1353

Dependent variable: d_spot

	coefficient	std. error	t-ratio	p-value	
d_spot_1lag	-1.32794	0.0512127	-25.93	5.91e-147	***
d_spot_2lag	-0.706064	0.0586757	-12.03	2.73e-33	***
d_spot_3lag	-0.277666	0.0569831	-4.873	1.10e-06	***
d_spot_4lag	-0.279929	0.0471507	-5.937	2.93e-09	***
d_futures_um_1lag	1.52276	0.0497675	30.60	3.98e-203	***
d_futures_um_2lag	0.635640	0.0581071	10.94	8.25e-28	***
d_futures_um_3lag	0.300344	0.0566317	5.303	1.14e-07	***
d_futures_um_4lag	0.293148	0.0474853	6.173	6.75e-10	***
Mean dependent var	0.251566	S.D. dependent var	143.9811		
Sum squared resid	7.25e+08	S.E. of regression	138.3936		
Uncentered R-squared	0.076282	Centered R-squared	0.076279		
F(8, 37862)	390.8355	P-value(F)	0.000000		
Log-likelihood	-240434.2	Akaike criterion	480884.3		
Schwarz criterion	480952.6	Hannan-Quinn	480906.0		

Listing 2.1: Fit to the first 90% of our March to December 2021 data set

Initial inspection of the model looks good: similar R² (slightly higher = 0.076) of fit as from the full data set, similar decaying back in time behavior of the model coefficients, same sign of coefficients as for the full data set model. All coefficients strongly significant as before.

Next, we merely evaluate the above model in the "future" with respect to the model time domain, e.g. points 39224 to 43581 (the last 10% of our data set). Results are shown in Figure 2.1 below.

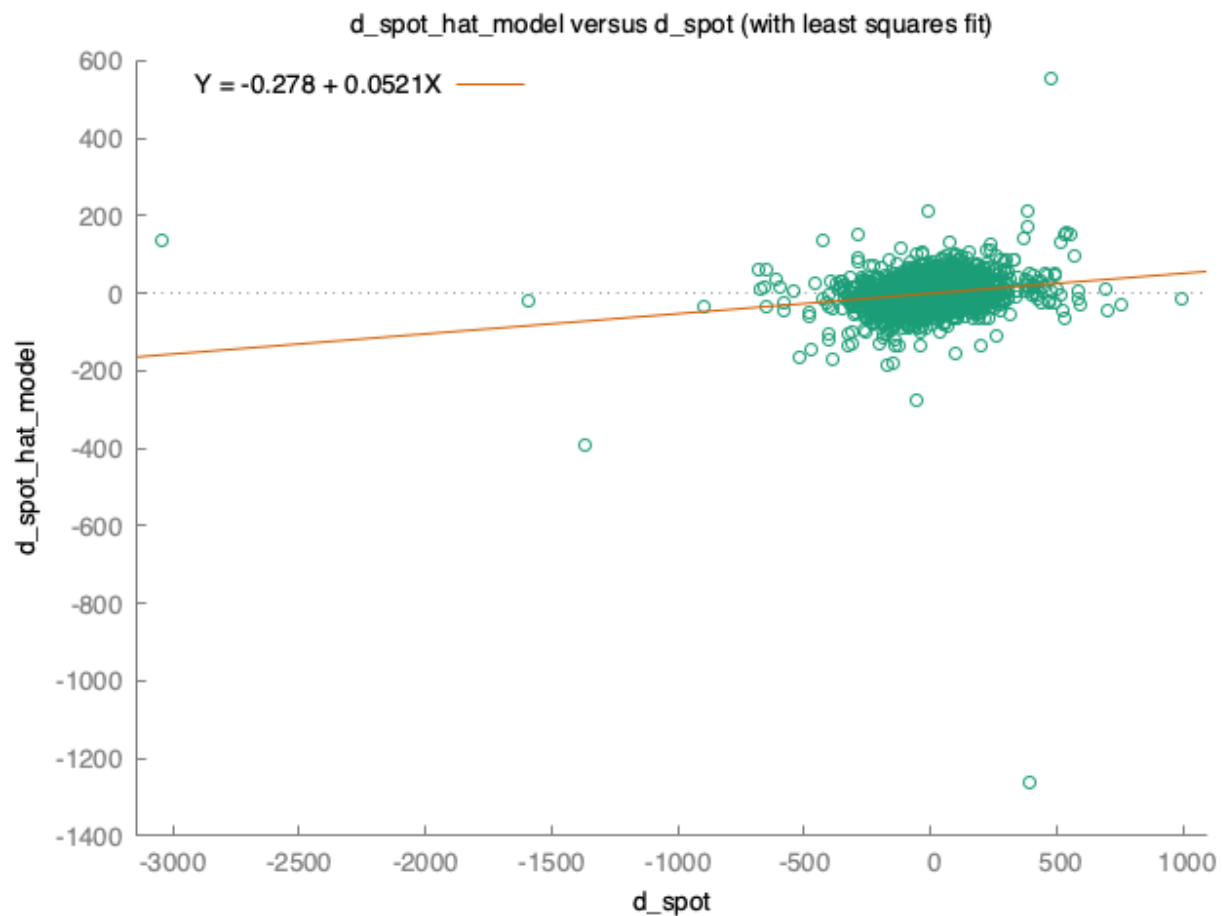


Figure 2.1: Out of sample evaluation (last 10% of points) from the model shown in Listing 2.1, $R^2 = 0.0354$, $N = 4358$, 40 missing observations

This R^2 "of forecast" is lower (by about half) than that of the R^2 "of fit" of 0.076 in our training set. However, as we saw in the first section of this paper, the story is not complete until we apply dead zone analysis to the forecasted points.

Applying the same dead zone analysis as in the first section of this paper, and using a dead zone tolerance of $\pm\$30$ (for example), we arrive at

696 directionally correct forecasts out of
1020 attempts (forecasts outside of the dead zone)

for a percent correct of 68.2%

This is approximately the "one forecast every 40 minutes" rate that we discussed earlier in this paper (1020 out of 4358 10 minute time intervals).

If we increase the dead zone to $\pm \$50$, the percent correct goes up to 75.1% (254/338) with 338 determinate forecast attempts out of 4358 10 minute intervals. This is about 1 in 13 ten minute intervals, which comes out to 130 minutes between forecast attempts (on average)... equivalent to about 11 forecasts per 24-hour day. Practically speaking, 11 forecasts per day at 75% directional accuracy seems like reasonable rates for an automated trade signal generator. There are enough forecasts to quickly establish a track record, yet not so many that it leads to an abundance of trades. However, even 11 trade recommendations per day may be too much for retail traders, so one could increase the dead zone more to reduce the frequency of recommendations (and likely also increase the percent correct, as we have seen), with the downside that it takes longer to establish a track record of recommended trades into the future.

Without getting too far into the use of a model like this yet, we start to note practical limitations of a potential trade signal generator product that might be built from such a model: once we have a model that forecasts "good enough" out-of-sample, we might not want it to spew recommendations to manual retail traders at too high of a rate, because it will be overwhelming. This suggests that when we perform refined model tuning, it might need to be done in the context of the model's use in a trade signal generator. Similar constraints exist for automated trading: if each trade has a transaction cost associated with it, we would prefer fewer trade recommendations at higher accuracy versus recommendations at a higher rate but lower accuracy.

Discussion

Examining Figure 2.1, we see 3 outlier forecasts (in the direction of the vertical axis) which may be driving down the R^2 OOS that we originally computed (compared to the in-sample R^2 of 0.076), because R^2 is particularly sensitive to outliers. The simple bullish/bearish binary outcome that results from applying the dead zone tolerance has no such sensitivity to outliers. A forecast is in the proper direction or it is not; it does not matter how much it "is not," it just gets counted as one wrong forecast if it is in the wrong direction.

We also see outliers in the modeled variable (X axis) here. Because of this, a regression model that is robust to outliers (unlike the simple OLS model we started with) might also be appropriate to try as a further refinement step.

Possible next steps for OOS checks

A more ideal split of known modeled data and checked / forecasted / withheld data could be estimated by cutting the data in various ways other than 90%/10%. It would also be useful to

know if it is best to put as much known data into the model as possible (back in time) or to build a model that uses a rolling window of known data (say, only pull data from 6 months back and cut off data older than that as time progresses forward; since, after all, the past is the past and market conditions "back then" may no longer be relevant). These types of model tunings (data size considerations, rolling windows) should be done in concert with other model tuning and refinement methods as described in the first section of this paper for best results. Another benefit of simpler models like this (versus machine learning methods) is that the model generation is fast (milliseconds), and so one can tune these model setup parameters such as data window sizes quicker than may be possible with longer-running ML models.

Section 2 Summary

A quick check of out-of-sample results for the simple two time series VAR model presented here verifies well. We get reasonable forecast rates and percent correct forecasts once we apply a dead zone to the raw model predictions. In spite of the promising OOS results, it remains to be seen if OOS forecasts of this quality connected to a trading system might result in realizable profits.